

HyNoC: A Hybrid Circuit-Switch/Wormhole Network-on-Chip for Distributed VLIW Computing on FPGA

Christophe Clienti
Open Hardware Project
CERN-OHL-P v2.0

<https://github.com/cclienti/verilog-ip>

Abstract—Network-on-Chip (NoC) architectures have become the standard interconnect fabric for many-core systems, yet most proposals face a fundamental trade-off between latency, area, and congestion management. This paper presents HyNoC (Hybrid Network-on-Chip), an open-source NoC architecture that combines circuit-switch path establishment with wormhole data transfer, targeting distributed computing systems built around VLIW processor cores on FPGA.

HyNoC employs source routing, where the complete path through the network is encoded in the packet header by the sender or statically at compile time, enabling both deterministic low-latency transfers and software-level hotspot avoidance without the area overhead of virtual channels. The router features a parallel round-robin arbiter (PRRA) with fixed grant latency, per-port independent clock domains, and support for both unicast and multicast routing.

We discuss the design rationale, the hybrid switching model, and positioning relative to prior NoC art, and argue that a richer topology combined with compiler-assisted static routing is a competitive alternative to virtual channels for FPGA-based distributed VLIW systems. Verilator co-simulation validates the analytical latency model and benchmarks the design at LLaMA 3 8B FFN up-projection scale: a four-master quadrant configuration achieves a $5\times$ throughput improvement over a single master with zero cross-quadrant link traffic. The complete RTL implementation is available as open-source hardware under the CERN-OHL-P v2 license.

Index Terms—Network-on-Chip, FPGA, source routing, wormhole switching, circuit switching, VLIW, parallel round-robin arbiter, open-source hardware.

I. INTRODUCTION

As the number of processing elements integrated on a single chip increases, scalable on-chip communication becomes a first-class design concern. Network-on-Chip (NoC) architectures have emerged as the standard answer to this challenge, replacing shared buses and point-to-point links with a structured, packet-switched interconnect fabric [1].

Existing NoC designs span a broad design space. Wormhole switching minimizes buffering by forwarding flits as soon as the header is decoded, but exposes the network to head-of-line blocking. Virtual channels (VCs) were introduced to mitigate this blocking [2], at the cost of duplicated buffers that can represent the dominant area contribution of a router [3]. Circuit switching provides fully dedicated, contention-free paths but requires path reservation before any data can flow.

HyNoC takes a different approach: it combines the path-dedication property of circuit switching—through a source-routed request/grant handshake at each hop—with the flit-level, FIFO-backed flow control of wormhole switching. This hybrid model is particularly well suited to FPGA targets, where block RAMs used as FIFOs are scarce resources, and to distributed VLIW computing workloads, where communication patterns are largely known at compile time and can therefore be routed statically to avoid congestion.

Originally inspired by the Hermes NoC [4], HyNoC departs significantly from its ancestor by introducing distributed arbitration, relative source routing with multicast support, per-port asynchronous clock domains, and a parallel round-robin arbiter that grants requests in a one or two cycles.

The contributions of this paper are:

- A precise characterization of HyNoC’s hybrid switching model and its implications for latency and area;
- A discussion of static route management as an alternative to virtual channels, supported by a combinatorial analysis of shortest-path counts in mesh and higher-dimensional topologies;
- An analysis of the scope and limitations of the no-VC design choice;
- An open-source RTL implementation targeting both simulation and FPGA synthesis, complemented by the Veriparse toolkit [5] for source-to-source elaboration of advanced RTL constructs and faster simulation;
- Verilator co-simulation results validating the analytical latency model and benchmarking a LLaMA 3 8B-scale distributed GeMV, demonstrating a $5\times$ throughput gain from a four-master quadrant configuration.

The remainder of the paper is organized as follows. Section II reviews related work. Section III describes the HyNoC architecture. Section IV details the hybrid switching model and the rationale for avoiding virtual channels. Section V discusses the target application domain. Section VI presents implementation details. Section VII reports simulation results, and Section VIII concludes.

II. RELATED WORK

NoC architectures span a broad design space, comprehensively covered in [6]; this section reviews the dimensions most

directly relevant to HyNoC’s design choices.

A. Switching Strategies

Three main switching strategies are found in the NoC literature. *Store-and-forward* switching requires a complete packet to be buffered at each router before forwarding, providing clean error isolation but imposing large buffer requirements and high latency [1]. *Wormhole switching* [7] pipelines flits across routers, dramatically reducing buffer depth requirements; latency depends on path length and contention. *Circuit switching* [8] reserves a dedicated path before transmission, eliminating mid-network contention at the cost of a setup overhead proportional to route length before the first data flit is delivered.

Hybrid approaches have been explored. *Æthereal* [9] combines guaranteed-throughput circuit-switched slots with best-effort wormhole traffic on the same physical links using time-division multiplexing. HyNoC takes a different hybrid point: it establishes a dedicated path through a hop-by-hop request/grant handshake (the circuit phase), then streams payload flits in wormhole fashion along that reserved path (the data phase). Unlike *Æthereal*, HyNoC does not multiplex traffic in time: once a path is open, it is held exclusively for the duration of the packet, and no other packet can pre-empt it on the same egress port. The setup latency — one request/grant round per hop — is paid once per packet and is bounded by the route length, which is acceptable for the long-burst transfers typical of VLIW inter-core communication.

B. Routing Algorithms

XY deterministic routing [1] is simple and deadlock-free on 2D meshes but provides only a single path between any pair of nodes. Adaptive routing [10] improves load balancing but requires more complex router logic and typically needs virtual channels to avoid deadlock.

Source routing encodes the complete route in the packet header at the sender. This approach, studied for NoCs in [11] and [12], shifts routing complexity from the router to the sending node or to an offline compilation step. The router itself becomes simpler: it only needs to decode the next hop from the current flit, forward it, and update the index. Source routing naturally supports any topology without per-router routing table storage, and allows the sender to choose among multiple paths. This compile-time route assignment property is particularly attractive for VLIW many-core systems, where communication patterns are statically known and can be scheduled to avoid contention (see Section V).

Source routing has also been adopted in production VLIW many-core NoCs: the Kalray MPPA2-256 uses a wormhole NoC with feed-forward source routing, where the complete route is encoded in the header flit and each router forwards based solely on a local offset extraction [13]. This production deployment validates the compile-time route assignment model that HyNoC targets.

HyNoC adopts source routing with a compact *relative* hop encoding: since a packet cannot exit from the same port it

entered, a router with N ports offers only $N - 1$ egress choices, encoded in $\lceil \log_2(N-1) \rceil$ bits per hop. This reduces both header overhead and crossbar complexity simultaneously.

C. Virtual Channels

Virtual channels (VCs), introduced by Dally [2], multiplex several logical channels over a single physical link, providing additional routing paths and breaking deadlock cycles. Their main drawback is area: each VC requires a dedicated buffer, and [3] shows that router area scales nearly linearly with VC count. On FPGAs, this overhead is particularly significant because both the application logic and the communication fabric draw from a shared pool of Block RAM (BRAM) and distributed LUT RAM resources: adding VCs directly reduces the memory budget available to the application.

Several works have explored VC reduction or elimination. Turn-model routing [14] achieves deadlock freedom without VCs by restricting the set of turns allowed in the routing graph. HyNoC’s approach—providing path diversity through richer topologies and compile-time route selection—aligns with this trend, and is analyzed quantitatively in Section IV.

D. NoC Arbitration

Arbiter design is a central concern in input-queued routers, where multiple ingress ports compete for the same egress [6]. Matrix arbiters evaluate all (ingress, egress) request pairs in parallel and can grant in a single cycle regardless of port count, but require $O(N^2)$ state and combinational logic, making them area-expensive for large routers. Separable allocators decompose the two-dimensional matching problem into two independent one-dimensional arbitration steps (one per ingress, one per egress), reducing area at the cost of potentially requiring multiple rounds to reach a stable matching [6]. Sequential round-robin FSMs scan requests cyclically, introducing up to $N - 1$ cycles of latency before granting a pending request.

HyNoC implements a *parallel round-robin arbiter* (PRRA): a set of $N - 1$ combinatorial LUT-based state functions that evaluate all pending requests simultaneously and jump directly to the winning state, regardless of how many ports are idle. The PRRA matches the single-cycle grant latency of a matrix arbiter for the common case, while its critical path does not grow with N since each LUT is evaluated independently. Fairness is maintained through a rotating priority identical to classical round-robin.

E. FPGA-Targeted NoCs

FPGA implementations impose constraints not present in ASIC designs: Block RAMs (BRAMs) are a finite, shared resource, so buffer-heavy NoC designs directly reduce the memory budget available to the application; clock domain crossing between the NoC fabric and the attached processing elements must be handled explicitly; and routing resources constrain achievable clock frequencies [15].

CONNECT [15] provides a parameterized, synthesizable NoC generator for FPGAs supporting configurable topologies, VC counts, and buffer depths, demonstrating that general-purpose FPGA NoCs are feasible. Its VC-inclusive design

trades BRAM for throughput robustness, which is appropriate when traffic patterns are unpredictable at design time.

Hoplite [16] introduced deflection routing on FPGA overlays, achieving very small area by eliminating input buffers entirely and routing contending flits onto an alternative output rather than buffering them. HopliteRT [17] extended Hoplite with a real-time traffic class, but deflection fundamentally introduces non-deterministic, potentially unbounded delivery latency for best-effort traffic — a property incompatible with the timing constraints of VLIW inter-core communication.

Xilinx’s Versal ACAP [18] embeds a hardened, fully programmable NoC in silicon alongside the FPGA fabric, demonstrating that on-chip network infrastructure has become a first-class concern even in commercial FPGA products. Its hardened implementation achieves high bandwidth with low latency, but is vendor-specific and not accessible as a soft IP.

Hermes [4], the direct ancestor of HyNoC, demonstrated low-area wormhole switching on FPGA with a 2D mesh topology and XY routing. Its asynchronous variant Hermes-A [19] explored per-port independent clock domains through asynchronous handshaking. HyNoC addresses the same requirement using synchronous dual-clock FIFOs, which bridge arbitrary clock domain boundaries at each ingress port without requiring asynchronous logic.

HyNoC targets a distinct point in this space: topology-agnostic source routing with bounded latency, no virtual channels, and LUT-RAM-based FIFOs that avoid BRAM consumption, suited to predictable VLIW workloads.

F. VLIW Many-Core Systems

VLIW processors expose multiple functional units to the compiler, which schedules operations explicitly in long instruction words [20]. This design philosophy transfers control complexity from hardware to software, producing simple, area-efficient cores well suited for replication. Signal processing, neural network inference, and scientific computing are representative workloads.

Distributed VLIW many-core systems—exemplified by the Kalray MPPA [21]—replicate VLIW cores connected through a dedicated NoC, with subsequent generations [13] adopting source routing to preserve deterministic latency guarantees across the fabric. In such systems, the communication graph is known at compile time: the compiler can assign source routes, schedule packet injections to avoid egress port collisions, and guarantee bounded communication latency on the critical path. This compile-time visibility makes HyNoC’s hybrid model particularly effective, as the bounded circuit-establishment latency (one PRRA grant per hop) feeds directly into the timing model used by the compiler to schedule inter-core transfers.

G. Positioning

Table I summarizes how HyNoC relates to the principal design axes discussed above.

TABLE I
POSITIONING OF REPRESENTATIVE NOCs

Design	Switching	Routing	VCs	Target
Hermes [4]	Wormhole	XY (dest.)	No	ASIC/FPGA
Æthereal [9]	TDM + Wormhole	Table	Yes	ASIC
CONNECT [15]	Wormhole	Configurable	Yes	FPGA
HopliteRT [17]	Deflection	XY (dest.)	No	FPGA
HyNoC	Circuit+Wormhole	Source (rel.)	No	FPGA

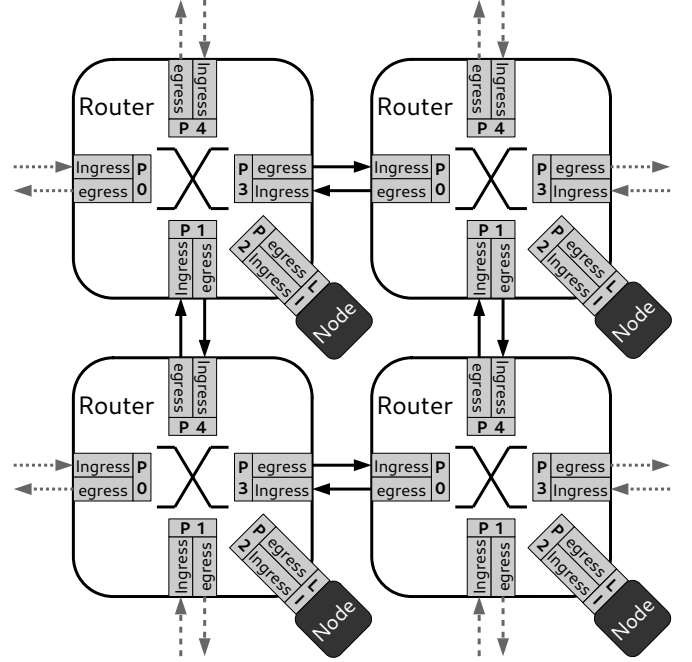


Fig. 1. Router interconnections overview.

III. HYNOC ARCHITECTURE

A. Overview

A HyNoC network is built by assembling routers with a configurable number of ports N (3 to 9 in the current implementation), where each flit is $K + 1$ bits wide ($K = \text{PAYLOAD_WIDTH}$, the extra bit being the stop bit). Each router port is full-duplex, composed of an ingress interface and an egress interface. Two neighboring routers are connected by cross-linking their egress and ingress interfaces. Processing nodes are attached through a *local interface*, which adds an extra output FIFO to decouple the node’s clock domain from the router.

The router implements a **full crossbar**: any ingress port can simultaneously communicate with any egress port, subject to arbitration. Multiple paths can be open concurrently inside the same router, allowing concurrent transfers between different ingress-egress pairs. Figure 1 shows the internal organization of 5-port routers and their interconnections.

B. Ingress Port

The ingress port receives flits from the egress of a neighboring router. It buffers incoming flits in a dual-clock FIFO (2^D

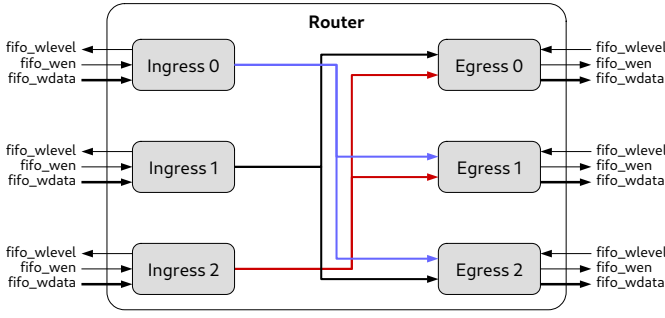


Fig. 2. Internal data path of a 3-port router.

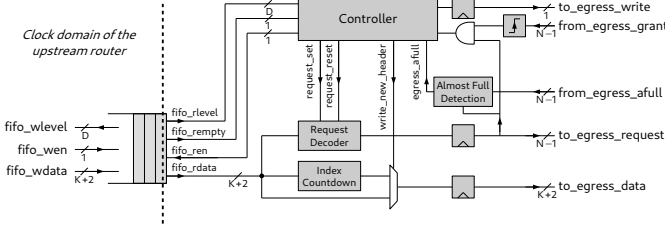


Fig. 3. Ingress port internal architecture: dual-clock FIFO and routing FSM.

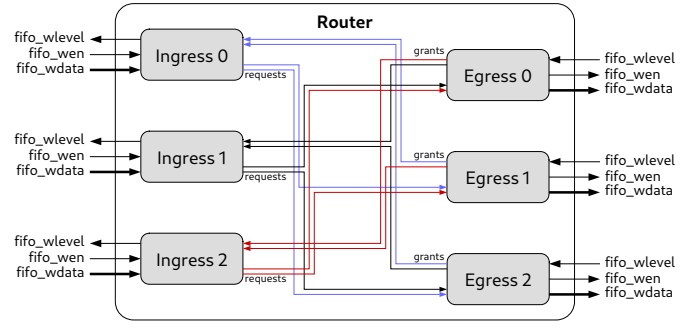


Fig. 4. Internal control path of a 3-port router.

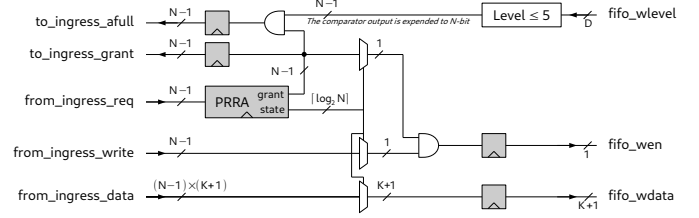


Fig. 5. Egress port internal architecture: PRRA arbiter and data multiplexer.

entries of $K + 1$ bits), which simultaneously handles clock domain crossing between the upstream router's clock domain and the local router domain.

Once flits are buffered, the FSM inspects the two most significant bits of the first flit to identify the flit type (routing or payload). It then extracts the next-hop egress port identifier, asserts the matching *request* bit among the $N - 1$ available egress ports, and waits for the corresponding *grant*. Upon grant, the ingress forwards the routing flit updated to the next hop (index decremented); if the index has already reached zero, the routing flit is consumed locally and not forwarded. Subsequent payload flits are streamed directly to the granted egress until the stop bit of the last flit is detected, at which point the channel is released.

Flow control is managed by monitoring both the downstream router's ingress FIFO level and the local ingress FIFO level. The ingress stalls when either FIFO approaches full, preventing flit loss. The internal data path of a 3-port router is illustrated in Figure 2.

C. Egress Port

The egress port is connected to $N - 1$ ingress ports via a $(K + 1)$ -bit wide data path. It arbitrates among concurrent write requests using the PRRA, which responds in one cycle when the port is idle, and grants one ingress at a time without starvation. Once a grant is issued, the data and write-enable signals from the winning ingress are routed directly to the downstream router's ingress FIFO. The downstream FIFO level is forwarded back to the granted ingress for flow control. The control path is shown in Figure 4: contrary to the data path, control signals are point-to-point between each ingress and each egress, avoiding any broadcast of control information.

D. Parallel Round-Robin Arbiter (PRRA)

The standard sequential round-robin FSM scans requests one by one in a fixed order, introducing up to $N - 1$ cycles of latency before granting a pending request (Figure 6). HyNoC replaces this with a **parallel round-robin arbiter (PRRA)**: a set of $N - 1$ combinatorial LUT-based state functions that evaluate all pending requests simultaneously and jump directly to the winning state, regardless of how many ports are idle.

The architecture instantiates $N - 1$ prra_lut modules in parallel (Figure 7), each pre-computing the next granted port for a different starting priority offset. The LUT corresponding to the current state is selected by a multiplexer; its output feeds the state register. The state transitions only when the currently granted request is de-asserted (end of packet) or when no request is active, locking the grant for the entire packet transfer and preventing spurious re-arbitration.

The arbiter responds in **one or two clock cycles** depending on the PIPELINE parameter: without pipelining the LUTs are purely combinatorial and the grant is issued in one cycle; with pipelining an extra register stage on the LUT outputs reduces the critical path at the cost of one additional cycle. The critical path does not grow with N because each LUT is independent and selected, not chained. Starvation is prevented by the rotating priority: after serving port k , priority rotates to port $k + 1 \bmod N$.

E. Clock Domain Architecture

Each router port operates in its own clock domain. The dual-clock FIFO at each ingress interface handles the crossing between the upstream port clock and the local router clock. For timing-critical designs, a single-clock mode

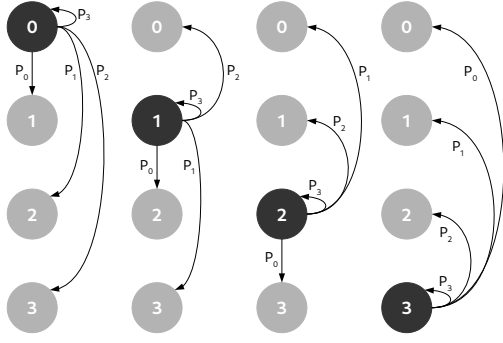


Fig. 6. Parallel Round-Robin FSM: direct state transitions avoid scanning idle ports.

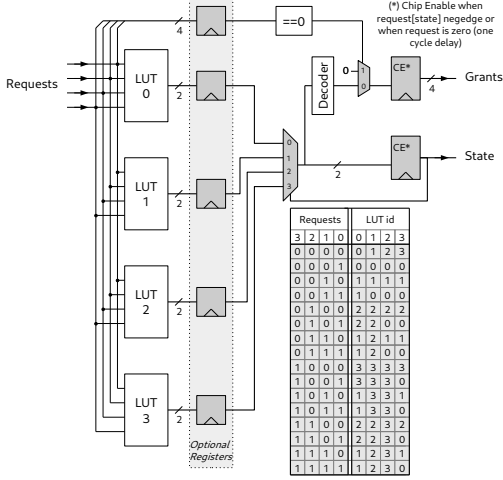


Fig. 7. PRRA architecture: $N - 1$ parallel LUTs, one per priority offset.

(SINGLE_CLOCK_ROUTER=1) is available, reducing crossing latency.

F. Local Interface

Any router port can be connected to a processing node instead of a neighboring router. The `hynoc_local_interface` module handles this attachment by instantiating an additional FIFO at the egress output. This FIFO serves two purposes: it fully decouples the node's clock domain from the router clock domain, and it smooths traffic bursts, preventing the node from becoming a bottleneck when it cannot consume data fast enough. From the router's perspective, the local interface is indistinguishable from a downstream router's ingress port, preserving the uniformity of the flow control mechanism across all port types.

AXI-Stream compatibility. The port-level handshake is deliberately modeled after the AXI-Stream protocol [22]: the write signal corresponds to TVALID, the full signal to the negation of TREADY, the data bus to TDATA, and the stop bit to TLAST. This mapping makes the interface immediately familiar to hardware designers working in the FPGA ecosystem. The only extension beyond standard AXI-Stream is the `fifo_level` sideband, which exposes the downstream buffer occupancy to

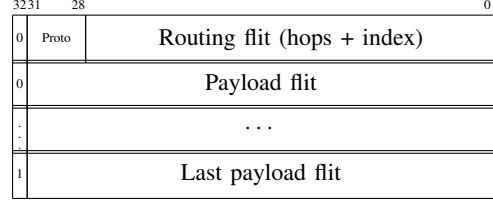


Fig. 8. Packet structure: routing flit(s) followed by payload flits. The MSB is the stop bit (0=intermediate, 1=last flit).

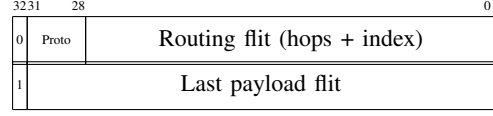


Fig. 9. Minimal HyNoC packet: one routing flit (stop bit = 0) followed by one payload flit (stop bit = 1). The routing flit stop bit must be zero since the ingress FSM rejects routing flits with stop bit set.

allow the sender to anticipate back-pressure before the full condition is reached.

G. First Layer Protocol

1) Packet and Flit Structure: The network operates on flits of $K + 1$ bits ($K = \text{PAYLOAD_WIDTH}$). The most significant bit is a *stop bit*: when set, it marks the last flit of a packet. A packet is built with at least one *Network Hops flit* (routing flit) followed by at least one *payload flit*, as shown in Figure 8. If multiple routing flits are present, intermediate routers treat all but the first as payload; each router consumes one routing flit from the front of the chain. The MSB of a routing flit is always zero, reserving space for future sub-type extensions. Figure 9 shows the smallest valid packet: one routing flit (stop bit must be zero) followed by one payload flit.

The routing flit carries a 4-bit *Proto* field (Table II):

TABLE II
SUPPORTED ROUTING PROTOCOLS

Proto	Routing method
4'b0000	Unicast circuit switch
4'b0001	Multicast circuit switch
4'b1000	XY routing (slot reserved, not yet implemented)
4'b1111	Forbidden

The proto value 4'b1000 is reserved for future XY routing support. XY routing would allow destination-addressed packets using (x,y) coordinates rather than explicit hop lists, enabling simpler packet construction at the cost of restricting the topology to 2D meshes. The ingress FSM currently flushes any packet carrying this proto value.

2) Unicast Source Routing: The Network Hops flit structure is shown in Figure 10. It carries H hop fields of $\lceil \log_2(N - 1) \rceil$ bits each, encoding the relative egress port ID at the next router, together with an *index* field initialized to $H - 1$. The index directly feeds the multiplexer selecting the active hop in the ingress port. It is decremented after each hop is consumed; if

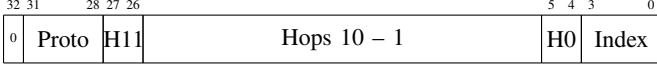


Fig. 10. Network Hops flit structure for a 5-port, 32-bit router (unicast, 12 hops of 2 bits, 4-bit index). The layout from MSB to LSB is: stop bit, Proto, Gap (if any), Hop $H - 1 \dots$ Hop 0, Index. The Gap absorbs unused bits between the Proto field and the first hop, and is zero in this particular configuration. The Index field (initialized to $H - 1$) selects the active hop.

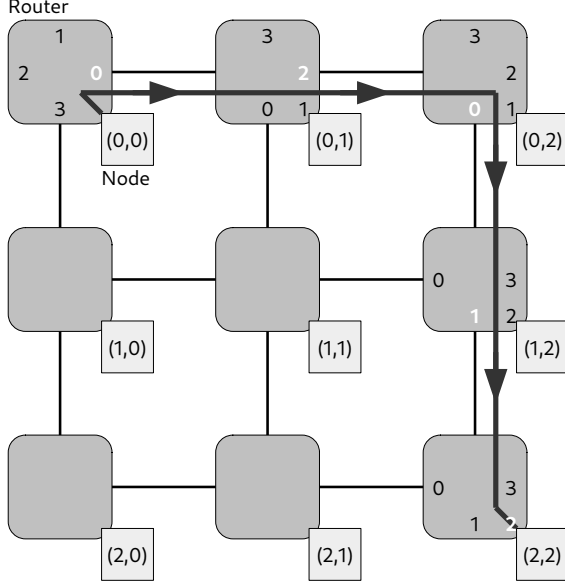


Fig. 11. Relative hop encoding for a 3×3 mesh network built with 5-port routers. The hop list $[0 \rightarrow 2 \rightarrow 0 \rightarrow 1 \rightarrow 2]$ establishes a path from node (0, 0) to node (2, 2). Each hop ID is relative to the ingress port: hop 0 denotes the next counterclockwise egress, excluding the ingress port itself.

the index is already zero before path opening, the routing flit is not forwarded downstream. The relative hop encoding on a 3×3 mesh of 5-port routers is illustrated in Figure 11.

Since a packet cannot exit through the same port it entered, each ingress port has only $N - 1$ reachable egress ports. Hop fields therefore require only $\lceil \log_2(N - 1) \rceil$ bits, which yields particularly compact encodings for the most common router sizes: a 3-port router encodes hops on 1 bit, a 5-port router on 2 bits, and a 9-port router on 3 bits. This reduction simultaneously shrinks the routing flit header overhead and the internal crossbar size, since the crossbar itself does not need to include the loopback path from ingress to its paired egress.

More precisely, the encoding constrains N to satisfy $2^{\lceil \log_2(N-1) \rceil} + 1 = N$, meaning only $N \in \{3, 5, 9\}$ are valid port counts. This constraint is enforced at elaboration time in the RTL. The relative hop ID is computed counterclockwise from the ingress port: hop 0 always refers to the next counterclockwise egress, and the ingress FSM resolves the relative ID into a one-hot physical egress request.

Given a flit of $K + 1$ bits, a 4-bit Proto field, and an INDEX_WIDTH-bit index field, the number of available hop bits

is $K - 3 - \text{INDEX_WIDTH}$, and the maximum hop count is:

$$H_{\max} = \left\lfloor \frac{K - 3 - \text{INDEX_WIDTH}}{\lceil \log_2(N - 1) \rceil} \right\rfloor \quad (1)$$

Table III gives the resulting values for the three valid router sizes in the default configuration ($K = 32$, INDEX_WIDTH = 4), for both unicast and multicast routing (which uses $N - 1$ bits per hop instead of $\lceil \log_2(N - 1) \rceil$). The wider multicast hop field halves the maximum hop count relative to unicast, reflecting the inherent trade-off between fanout expressiveness and path length.

TABLE III
MAXIMUM HOP COUNT PER ROUTING MODE ($K = 32$, INDEX_WIDTH=4)

Router	Unicast		Multicast	
	bits/hop	$H_{\max}$	bits/hop	$H_{\max}$
3-port	1	24	2	12
5-port	2	12	4	6
9-port	3	8	8	3

3) *Multicast Source Routing*: In multicast mode, each hop field is a bitmask of width $N - 1$, where each bit selects one egress port. The packet is duplicated and forwarded to all asserted egress ports simultaneously. Multiple routing flits with different multicast masks can be chained to address complex multicast trees. This is particularly useful for broadcast operations in VLIW many-core workloads.

IV. HYBRID SWITCHING MODEL AND DESIGN RATIONALE

A. The Hybrid Switching Model

HyNoC's switching model is best described as *circuit-switched wormhole*: it combines a **circuit-establishment phase** with a **wormhole data phase**.

In the **circuit-establishment phase**, the ingress port asserts a request to a specific egress port (identified from the routing flit) and waits for a grant. This request/grant handshake propagates along the route: once the first router grants the request, the ingress begins forwarding flits, which in turn trigger the same handshake at the next router. A dedicated forwarding path through the network is thus progressively opened, hop by hop.

In the **wormhole data phase**, payload flits are streamed flit-by-flit along the established path, regulated by FIFO-level flow control. The path remains dedicated to this packet until the stop bit is observed, ensuring in-order, contention-free delivery once the channel is open.

This combination provides the *low buffering* advantage of wormhole switching (only the ingress FIFO is needed, no full packet storage) while offering the *path dedication* advantage of circuit switching (no mid-network arbitration contention during payload transfer). The trade-off is a path-establishment latency proportional to the route length before the first payload flit is delivered at the destination.

B. End-to-End Latency Model

Let H denote the number of hops from source to destination (including the final local hop), and P the number of payload flits in the packet. The end-to-end latency L decomposes into two additive terms.

Setup phase. The routing flit propagates hop by hop; at each hop the ingress FSM reads the flit from the FIFO, asserts a request, and waits for a grant. Per-hop setup latency is:

$$T_{\text{hop}} = T_{\text{FIFO}} + 1 + T_{\text{PRRA}} \quad (2)$$

where T_{FIFO} is the FIFO read latency (1 cycle in single-clock mode; ≈ 3 cycles in dual-clock mode due to the gray-code CDC synchronisation pipeline [7]), the $+1$ accounts for the FSM decode cycle, and $T_{\text{PRRA}} \in \{1, 2\}$ is the arbiter grant latency controlled by the PIPELINE parameter.

Transfer phase. Once the path is established, P payload flits are pipelined at one flit per hop: the first flit arrives at the destination after H cycles, and the last after $H + P - 1$ cycles.

The total latency is therefore:

$$L = H \cdot (T_{\text{FIFO}} + 2 + T_{\text{PRRA}}) + P - 1 \quad (3)$$

In the minimum configuration (single-clock, PIPELINE=0): $L = 4H + P - 1$. In the maximum configuration (dual-clock, PIPELINE=1): $L = 7H + P - 1$. Both bounds are tight and deterministic: L depends only on the statically-known route length H and packet size P , with no congestion-dependent term once the path is open.

C. Static Route Management as an Alternative to Virtual Channels

Virtual channels were introduced primarily to: (1) provide additional logical paths to avoid deadlock; and (2) reduce head-of-line blocking under congestion. HyNoC addresses both goals differently.

Deadlock avoidance. Deadlock in wormhole networks arises from cyclic dependencies in the channel dependency graph (CDG) [7]: a set of flits each waiting for a channel held by the next forms an unbreakable cycle. With source routing, the complete path is fixed at injection time by the sender. Deadlock freedom therefore reduces to a property of the *route set*: if every injected route follows a routing function whose CDG is acyclic — dimension-order routing being the canonical example — no cycle can form, regardless of traffic load. This analysis is performed once, statically, by the compiler or network configurator; the router itself carries no deadlock-prevention logic. The guarantee is strictly stronger than VC-based approaches, where deadlock freedom depends on runtime routing decisions and requires the router to maintain channel state across multiple virtual queues.

Congestion avoidance is addressed at two levels. First, the compiler or runtime system assigns routes across multiple available shortest paths, distributing traffic away from known hotspots. The number of shortest paths between two nodes in an $R \times C$ mesh is a classical combinatorial result [23]: each path consists of a fixed sequence of moves along each

dimension, taken in any order, and the count is given by the multinomial coefficient:

$$p = \binom{(R-1) + (C-1)}{R-1} = \frac{[(R-1) + (C-1)]!}{(R-1)!(C-1)!} \quad (4)$$

This generalizes to a K -dimensional network $n_1 \times n_2 \times \dots \times n_K$ as:

$$p = \frac{\left[\sum_{i=1}^K (n_i - 1) \right]!}{\prod_{i=1}^K (n_i - 1)!} \quad (5)$$

Note that (4) and (5) give the number of shortest paths between the *most distant* node pair (corner-to-corner). For an arbitrary pair separated by (d_x, d_y) hops the count is $\binom{d_x + d_y}{d_x}$, which is lower. Table IV reports corner-to-corner values for representative 2D mesh topologies.

TABLE IV
SHORTEST PATH COUNTS IN 2D MESH NoCs (CORNER-TO-CORNER)

Topology	Routers	Shortest paths	Path length
2×2	4	2	2
4×4	16	20	6
8×8	64	3 432	14
16×16	256	155 117 520	30

Second, for applications with more dynamic traffic, the network topology can be enriched with additional relay routers (without local interfaces), increasing path diversity proportionally to the added area.

D. Area Comparison with Virtual Channels

Following [3], router area scales linearly with the number of VCs (each VC requires an additional input buffer per port). An 8×8 mesh with 4 VCs consumes the equivalent area of a 16×16 mesh without VCs, yet the latter offers over $45,000\times$ more shortest paths between corner nodes (Table IV: 155,117,520 vs. 3,432). On FPGA, where block RAMs are scarce, minimizing buffer count is a primary concern.

The VC area model is itself optimistic in two respects: (1) the crossbar logic and VC scheduler also grow with VC count; (2) VC path diversity is an upper bound, since in practice the VC is assigned at the source and remains fixed for the entire path, providing less diversity than a true additional physical dimension. Conversely, the physical router area model does not capture link wiring cost, which may be relevant in ASIC contexts.

E. Head-of-Line Blocking and Mitigations

Once a channel is granted, it remains dedicated until the stop bit of the last flit is received. A long packet therefore delays other packets competing for the same egress port. Two complementary mitigations are available in HyNoC:

- 1) **Network enrichment:** adding more routers distributes traffic across more egress ports, reducing the probability that multiple flows compete for the same port. The cost is

increased end-to-end *latency* due to additional hops—this is the primary performance trade-off, not throughput.

- 2) **Multiple local interfaces:** a node can expose several local interfaces connected to different router ports, providing as many independent logical channels as needed. This replicates the path diversity benefit of VCs without duplicating internal router buffers. It is the recommended approach when a node requires simultaneous, non-blocking communication with multiple peers.

F. Scope and Limitations

The topology-based approach is most effective when traffic patterns are known at compile time and routes can be assigned statically. When traffic is entirely unpredictable at design time and topology enrichment is not feasible, virtual channels remain the more appropriate solution. HyNoC targets systems—distributed VLIW processors on FPGA—where compile-time route assignment is natural and FPGA resource budgets make buffer minimization a primary concern.

V. TARGET APPLICATION: DISTRIBUTED VLIW COMPUTING

A. VLIW Cores and Compile-Time Scheduling

VLIW processors expose multiple functional units to the compiler, which schedules operations explicitly in long instruction words [20]. This design philosophy shifts control complexity from hardware to software, resulting in simple, area-efficient cores well suited for replication in many-core arrays. Signal processing, scientific computing, and neural network inference kernels are representative workloads.

In a distributed VLIW many-core system, each core executes a portion of a data-flow graph, passing intermediate results to neighboring cores through the NoC. The communication graph is typically known at compile time, enabling the compiler to:

- 1) Assign each communication a source route through the NoC;
- 2) Schedule packet injections to avoid simultaneous contention on the same egress port;
- 3) Guarantee bounded communication latency for critical paths.

This compile-time visibility makes HyNoC’s hybrid model particularly effective: the circuit-establishment latency is paid once per packet and is deterministic (bounded by route length and PRRA grant latency), and the wormhole payload transfer proceeds without further arbitration overhead.

B. Integration with the Local Interface

Each VLIW core connects to the NoC through a local interface, which presents a simple FIFO-based write/read API. The core writes routing flits followed by payload flits into the local interface’s ingress FIFO; the local interface manages all handshaking with the router. On the receive side, incoming flits are deposited in the egress FIFO and read by the core at its own pace. The dual-FIFO structure fully decouples the core’s clock domain from the router domain.

The `hynoc_stream_writer` and `hynoc_stream_reader` modules are simulation utilities that generate and check random packet streams, respectively. They serve as traffic generators in the testbench environment, verifying the correctness of the NoC under various load conditions.

C. Multicast for Broadcast Operations

VLIW many-core applications frequently require broadcast operations: distributing a coefficient matrix, a lookup table, or a control token to multiple cores simultaneously. HyNoC’s multicast routing protocol supports this natively: a single packet with a multicast routing flit is replicated at each router toward all targeted egress ports, without requiring the sender to issue multiple unicast packets. This reduces network load and sender overhead for one-to-many communication patterns common in data-parallel VLIW workloads.

VI. IMPLEMENTATION

A. Repository Organization

HyNoC is part of a larger open-source IP library hosted at <https://github.com/cclienti/verilog-ip>. The repository is organized into two complementary layers under the `hw/` directory:

- `hw/lib/`: a collection of reusable, standalone Verilog IP cores covering arithmetic, memories, FIFOs, and arbiters. Each module ships with its own parameterized RTL and self-checking testbench.
- `hw/network/hynoc/`: the HyNoC subsystem, which builds on top of `hw/lib/` and is described in detail in this paper.

HyNoC directly instantiates three library modules: `dclkfifolut` (dual-clock FIFO mapped to distributed LUT RAM, used in `hynoc_ingress` and `hynoc_local_interface`), `sclkfifolut` (single-clock variant, used in single-clock mode), and `prra` together with its `prra_lut` sub-module (the parallel round-robin arbiter, used in `hynoc_egress`). These components are independently verified and reusable outside HyNoC. Relying on shared library primitives rather than vendor-specific FIFO or arbitration IP is what makes the design portable across FPGA families.

B. RTL Organization

The HyNoC RTL is written in Verilog and organized as follows:

- `hynoc_ingress`: ingress port (dual-clock FIFO via `dclkfifolut/sclkfifolut`, routing FSM, unicast and multicast protocol decoder);
- `hynoc_egress`: egress port (data multiplexer, PRRA via `prra/prra_lut`);
- `hynoc_router_base`: parameterized router base (crossbar wiring, port instantiation);
- `hynoc_router_3p` / `hynoc_router_5p`: 3-port and 5-port router instances with self-checking testbenches;
- `hynoc_local_interface`: local interface for node attachment (output FIFO via `dclkfifolut/sclkfifolut`);

- `hynoc_stream_reader / hynoc_stream_writer`: simulation traffic generators used in testbenches to inject and check random packet streams.

The design is parameterized by: payload width (`PAYLOAD_WIDTH`), FIFO depth (`LOG2_FIFO_DEPTH`: 2 to 64 entries), number of ports (`NB_PORTS`: 3 to 9), index width (`INDEX_WIDTH`), optional multicast routing (`ENABLE_MCAST_ROUTING`), and single-clock mode (`SINGLE_CLOCK_ROUTER`).

C. Simulation and Verification

Each module is delivered with a self-checking testbench. The 3-port router testbench exercises both unicast and multicast scenarios over a 2-router, 4-node topology. The 5-port router testbenches cover five distinct traffic scenarios, including simultaneous multi-path transfers that verify the PRRA's starvation-free behavior.

Simulation is supported under Icarus Verilog and ModelSim. Verilator is integrated in the build infrastructure both for linting (`-lint-only`) and for C++ co-simulation: a multi-instance Verilator model is used in Section VII to characterize end-to-end latency and link utilization on a 4×4 mesh. Waveform inspection is automated using Wavedisp [24], a Python module that generates simulator-specific signal configuration scripts from a single source description, targeting GTKWave, ModelSim, and Riviera-PRO.

D. FPGA Target

Table V reports post-place-and-route results on two Xilinx 7-series devices: a Kintex-7 (`xc7k160t-fbg484`, speed grade -2) targeting 370 MHz and a Zynq-7020 (`xc7z020clg484`, speed grade -1) targeting 200 MHz. Both runs use 32-bit payload and 32-entry single-clock FIFOs (`sclkfifo_lut`). **WNS** (Worst Negative Slack) is the tightest setup slack across all register-to-register timing paths; a positive value means the target is met, and $f_{\max} = 1 / (T_{\text{clk}} - \text{WNS})$.

TABLE V
P&R RESULTS ON TWO XILINX 7-SERIES DEVICES (32-BIT, 32-ENTRY `sclkfifo_lut`; 24 LUT-RAM/PORT FOR INGRESS FIFOs). WNS IN NS; $f_{\max}$ IN MHZ.

Router	LUT (log.)	LUT (RAM)	FFs	Kintex-7 -2 (370 MHz)		Zynq-7020 -1 (200 MHz)	
				WNS (ns)	$f_{\max}$ (MHz)	WNS (ns)	$f_{\max}$ (MHz)
3-port	253	72	327	+0.079	381	+0.034	201
5-port	596	120	590	+0.127	389	+0.101	204
7-port	1259	168	889	+0.070	380	+0.032	201
9-port	1806	216	1215	+0.003	371	+0.034	201

LUT-RAM usage is exactly 24 per port in every configuration, reflecting the fixed-depth FIFO at each ingress; no block RAM is consumed. LUT counts are device-independent to within 1% (synthesis tool variation). Logic LUTs grow superlinearly with port count, driven by the PRRA whose table size scales as 2^{N-1} per egress port. FFs grow nearly linearly (109 to 135 per port), consistent with the per-port register cost of the

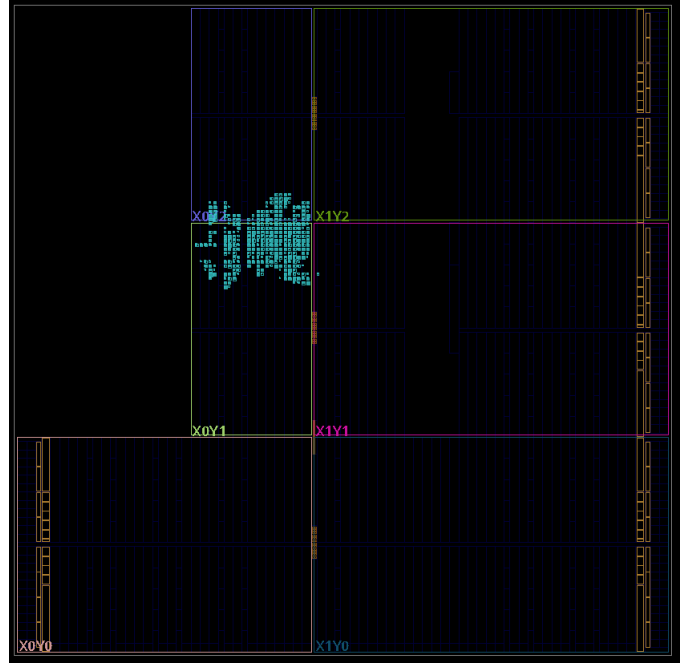


Fig. 12. Post-place-and-route floorplan of the 5-port, 32-bit router on the Zynq-7020 (`xc7z020clg484`, speed grade -1 , 200 MHz target).

ingress FSM and FIFO control. All four port counts meet their respective frequency targets on both devices, with the 5-port configuration consistently achieving the best timing margin.

Figure 12 shows the post-place-and-route floorplan of the 5-port router on the Zynq-7020. The compact, regular placement reflects the uniform per-port structure: each ingress column maps to a distinct LUT-RAM cluster, with the PRRA logic concentrated at the centre of the bounding box.

The design has been validated on both Intel (Altera) and AMD (Xilinx) FPGA families; reference board support is included for the DE0-Nano (Cyclone IV) and the ZedBoard (Zynq-7000). Both `dclkfifo_lut` (dual-clock) and `sclkfifo_lut` (single-clock) map exclusively to distributed LUT RAM, avoiding dependency on vendor-specific FIFO IP and ensuring portability across FPGA families.

The AXI-Stream-compatible port interface (see Section III) simplifies integration with standard FPGA IP cores. On Zynq devices for instance, the HyNoC local interface connects directly to the Processing System AXI-Stream DMA engine with only a thin adapter to bridge the `fifo_level` sideband. This reduces integration effort and allows reuse of existing AXI-Stream verification infrastructure.

E. RTL Elaboration with Veriparse

HyNoC makes use of advanced Verilog constructs — in particular, initial blocks containing for loops and integer arithmetic — to pre-compute the PRRA lookup tables at elaboration time. While this is perfectly legal Verilog, some FPGA toolchains have limited support for such constructs in synthesizable RTL.

Before (parameterized):

```

module hynoc_router_5p
  #(parameter PAYLOAD_WIDTH  = 32,
    parameter LOG2_FIFO_DEPTH = 5, ...)
  (...
    input wire [PAYLOAD_WIDTH:0] port0_ingress_data,
    output wire [LOG2_FIFO_DEPTH:0] port0_ingress_fifo_level,
    ...);

```

After veriflat (parameters inlined):

```

module hynoc_router_5p
  (...
    input wire [32:0] port0_ingress_data,
    output wire [5:0] port0_ingress_fifo_level,
    ...);

```

Fig. 13. veriflat resolves parameterized port widths into constants.

Veriparse [5] is a companion source-to-source transformation toolkit that addresses this issue. Its *veriflat* tool performs a sequence of elaboration-time passes on the full module hierarchy: *parameter and localparam inlining* substitutes all parameter values at instantiation sites; *constant folding* and *expression evaluation* reduce symbolic expressions to integer literals; *loop unrolling* expands for, while, and repeat constructs; *generate-block elaboration* resolves generate/endgenerate sections; *variable folding* evaluates initial blocks whose control flow is entirely determined by inlined constants; and *dead-code elimination* prunes branches that can never be taken after constant propagation. The *module flattener* then inlines the full instantiation hierarchy into a single, parameter-free module. The resulting RTL contains only plain assignments, registers, and combinational logic, with no remaining parametric expressions — portable across any standard Verilog toolchain.

Figure 13 illustrates parameter inlining and constant folding on the module boundary: the parameterized `hynoc_router_5p` declaration, whose port widths are symbolic expressions of `PAYLOAD_WIDTH` and `LOG2_FIFO_DEPTH`, becomes a flat declaration with all widths resolved to integer literals. The 5-port, 32-bit configuration yields `[32:0]` data buses and `[5:0]` FIFO level signals in the output.

Figure 14 shows a complementary transformation class supported by *veriflat*: variable folding combined with loop unrolling. The initial block of `prra_lut` — which computes the round-robin priority table through nested for loops and modular arithmetic over inlined constants — is fully evaluated at elaboration time; the resulting initial block contains only explicit constant assignments.

The flattened output also benefits simulation: a single-module design with no hierarchy reduces simulator overhead and can significantly speed up large testbench runs. Veriparse is available at <https://github.com/ccienti/veriparse> under the LGPLv3 license.

F. License

HyNoC has been in development since 2013, as reflected in the copyright headers of every RTL source file. The source

Before (algorithmic initial block):

```

initial begin
  lut[0] = STATE_OFFSET[LOG2_WIDTH-1:0];
  for (j=1; j<lut_length; j=j+1) begin
    lut_index = j[WIDTH-1:0];
    value = -1;
    for (k=0; k<WIDTH; k=k+1) begin
      l = (k + STATE_OFFSET + 1) % WIDTH;
      if (lut_index[l] == 1'b1)
        if (value == -1) value = 1;
    end
    lut[j] = value[LOG2_WIDTH-1:0];
  end
end

```

After veriflat (pre-computed, STATE_OFFSET=0):

```

initial begin
  lut[0]=2'd0; lut[1]=2'd0; lut[2]=2'd1; lut[3]=2'd1;
  lut[4]=2'd2; lut[5]=2'd2; lut[6]=2'd1; lut[7]=2'd1;
  lut[8]=2'd3; lut[9]=2'd3; lut[10]=2'd1; lut[11]=2'd1;
  lut[12]=2'd2; lut[13]=2'd2; lut[14]=2'd1; lut[15]=2'd1;
end

```

Fig. 14. veriflat evaluates the `prra_lut` initial block: nested loops and modular arithmetic are replaced by 16 explicit constant assignments (4-port arbiter, `STATE_OFFSET=0`).

code was first published in 2019; the repository was formally placed under the **CERN-OHL-P v2** license [25] in 2026. This permissive open hardware license allows use, modification, and redistribution with no copyleft obligation on the end product. The complete source, including RTL, testbenches, documentation, and build infrastructure, is available at: <https://github.com/ccienti/verilog-ip>.

VII. EVALUATION: DISTRIBUTED MATRIX-VECTOR MULTIPLICATION

We present three Verilator [26]-based simulations on a 4×4 HyNoC mesh, each targeting a different evaluation goal:

- 1) **Latency model validation.** A lightweight GeMV (General Matrix-Vector multiplication, $y = Ax$) with a 16×4 matrix produces clean per-hop measurements that validate the analytical model of Section IV.
- 2) **LLaMA 3 8B FFN benchmark (single master).** A full-scale GeMV at LLaMA 3 8B FFN up-projection dimensions (14336×4096) with Q8_0 weights and BF16 activations demonstrates the network under realistic AI inference workload.
- 3) **Four-master throughput scaling.** The same LLaMA 3 8B-scale benchmark is replicated across four corner masters, each owning a 2×2 quadrant, to quantify the speedup from distributed injection.

All three benchmarks share the same 16-router 4×4 mesh and cover the full communication cycle: data scatter from the master(s), computation at worker nodes, and result gather.

A. Network Topology

The network consists of 16 `hynoc_router_5p` instances arranged as a 4×4 2D mesh, as shown in Figure 15. Routers are

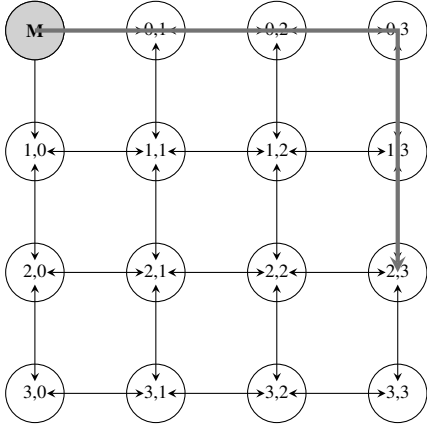


Fig. 15. 4×4 mesh topology. **M** = master at $(0,0)$; remaining nodes are workers at (r,c) . Bidirectional links are full-duplex (independent ingress and egress). The gray arrow illustrates the XY route from the master to worker $(2,3)$: three East hops followed by two South hops, then a final local hop ($H = 6$).

addressed by (r,c) with $r \in \{0,1,2,3\}$ (row, top-to-bottom) and $c \in \{0,1,2,3\}$ (column, left-to-right). Port convention per router: 0=Local, 1=East, 2=South, 3=West, 4=North. Each router's local port (port 0) attaches a processing node; neighboring routers are cross-linked on their directional ports (e.g., router (r,c) port 1 connects to router $(r,c+1)$ port 3).

The master node resides at $(0,0)$. It sequentially distributes data to all 15 worker nodes and collects their results.

B. Computation and Communication Model

The benchmark computes $\mathbf{y} = \mathbf{A}\mathbf{x}$ with $\mathbf{A} \in \mathbb{R}^{16 \times 4}$ and $\mathbf{x} = [1, 2, 3, 4]^T$. Worker (r,c) is responsible for one scalar inner product: $y_{4r+c} = \langle \mathbf{A}_{4r+c}, \mathbf{x} \rangle$.

Forward packet (master to worker (r,c)): one routing flit, one tag flit, four flits for the matrix row $\mathbf{A}_{4r+c}$, and four flits for $\mathbf{x}$, totaling ten flits. After the routing flit is consumed by the network, the worker receives $P_{\text{fwd}} = 9$ payload flits.

Return packet (worker to master): one routing flit, one tag flit, and one result flit. The master receives $P_{\text{ret}} = 2$ payload flits.

Workers compute the inner product instantaneously, so the total simulation time is dominated by communication latency and the sequential injection schedule at the master.

Route construction. Hop lists are built following dimension-order (XY) order: East/West moves bring the packet to the target column, North/South moves bring it to the target row, and a final local hop delivers it. This eliminates cyclic channel dependencies and guarantees deadlock freedom [7]. All packets use standard unicast source routing (proto 4'b0000); the dedicated XY proto field (4'b1000) reserved in Table II is not exercised in these experiments. A route from (r_0, c_0) to (r_1, c_1) requires

$$H = |c_1 - c_0| + |r_1 - r_0| + 1 \quad (6)$$

hops, where the +1 accounts for the final local hop at the destination. Throughout this section H always counts all hops

including that local hop. For the master at $(0,0)$, this simplifies to $H = r + c + 1$. Routes are encoded as source-routing flits using the relative hop encoding described in Section III.

C. Simulation Infrastructure

The Verilator C++ model instantiates 16 `Vhynoc_router_5p` objects sharing one `VerilatedContext`. Inter-router links are modeled as one-cycle latches: egress signals are captured after each rising clock edge and applied to the neighbor's ingress before the next rising edge. This one-cycle propagation delay is physically accurate for a registered synchronous interconnect.

Each router drives a single clock (`router_clk` and all `portX_ingress_clk` toggled together). In dual-clock mode (`SINGLE_CLOCK_ROUTER=0`), the ingress dual-clock FIFOs internally use gray-code CDC synchronization even when source and destination clocks are the same signal; in single-clock mode (`SINGLE_CLOCK_ROUTER=1`), the gray-code pipeline is bypassed.

D. Latency Results

Table VI reports the *return-path* latency for each hop count H in both configurations. Return-path measurements are free of source-queuing effects (each worker sends exactly one result as soon as computation completes), yielding clean linear fits.

TABLE VI
RETURN-PATH LATENCY ($P = 2$) VS. HOP COUNT. DUAL = DUAL-CLOCK FIFO (`SINGLE_CLOCK_ROUTER=0`); SINGLE = SINGLE-CLOCK FIFO (`SINGLE_CLOCK_ROUTER=1`); Δ = DUAL - SINGLE (CYCLES).

H	Simulated (cy.)			Theory (3)		
	Dual	Single	Δ	Dual	Single	Δ
2	25	19	6	13	9	4
3	37	28	9	19	13	6
4	49	37	12	25	17	8
5	61	46	15	31	21	10
6	73	55	18	37	25	12
7	85	64	21	43	29	14

The measured latencies satisfy exact linear fits:

$$L_{\text{dual}} = 12H + P - 1 \quad (7)$$

$$L_{\text{single}} = 9H + P - 1 \quad (8)$$

Both fits are consistent with the model of (3): latency scales linearly with hop count, with a per-hop coefficient determined by T_{FIFO} , the FSM decode cycle, and T_{PRRA} . The theoretical minimum slopes (`PIPELINE=0`) are $6H$ for dual-clock ($T_{\text{FIFO}} \approx 3$) and $4H$ for single-clock ($T_{\text{FIFO}} = 1$). The simulation slopes are higher for two reasons: (i) the gray-code synchronization pipeline incurs additional latency when source and destination clocks are the same signal, as is the case in this single-clock Verilator environment; and (ii) the one-cycle inter-router link latch adds one cycle per hop. Both effects vanish in a real FPGA deployment where clock domains differ and wiring is combinatorial within the same clock cycle.

The critical result is the **improvement margin**: theory predicts a difference of $2H$ cycles between the two modes;

simulation measures $3H$, the extra cycle per hop being exactly the CDC overhead eliminated by single-clock operation. The reduction is constant and perfectly linear, confirming the deterministic nature of the HyNoC latency model.

The total simulation time is **391 cycles** in dual-clock mode and **349 cycles** in single-clock mode (-10.7%). All 16 computed results are correct in both runs.

E. Link Utilization

Table VII summarizes link utilization over the 391-cycle dual-clock run. Traffic is inherently asymmetric: the master sends all forward packets via its East port first, concentrating load on the East-bound links of row 0.

TABLE VII
LINK UTILIZATION (%) BY DIRECTION (DUAL-CLOCK, 391 CYCLES).
SEG. k : k -TH LINK FROM MASTER (0, 0).

Direction	Seg. 1	Seg. 2	Seg. 3
→East, row 0	30.7	20.5	10.2
→West, row 0	2.3	1.5	0.8
→South, col 0	7.7	5.1	2.6
→North, col 0	9.2	—	—
Overall average: 3.3%			

East-bound utilization halves at each column of row 0, consistent with XY routing: each router delivers one packet to its local worker and forwards the rest eastward. The overall average of 3.3% reflects the sequential master bottleneck: with a single sender issuing one packet at a time, most links are idle.

F. LLaMA 3 8B FFN Benchmark — Single Master

To assess HyNoC under a realistic AI inference workload we scale the benchmark to the LLaMA 3 8B [27] FFN up-projection layer. In transformer models, the Feed-Forward Network (FFN) consists of two linear projections with a nonlinearity in between; the *up-projection* is the first of these, expanding the representation from the model dimension $D_{\text{in}} = 4096$ to the intermediate dimension $D_{\text{out}} = 14336$. The benchmark computes $\mathbf{y} = \mathbf{W}\mathbf{x}$ with $\mathbf{W} \in \mathbb{R}^{14336 \times 4096}$ and $\mathbf{x} \in \mathbb{R}^{4096}$.

Weights are stored in the Q8_0 quantization format used by llama.cpp [28]: each weight is an 8-bit signed integer, and one FP16 (16-bit floating-point) scale factor is shared per block of 32 weights to recover approximate FP32 values at decode time. Mapping to flits: the FP16 scale occupies the lower 16 bits of one 32-bit flit (upper 16 bits zero), and the 32 int8 weights are packed four per flit, requiring 8 flits — hence 9 flits per Q8_0 block. Activations use the BF16 format (Brain Float 16: 1 sign bit, 8 exponent bits, 7 mantissa bits), packed two values per 32-bit flit. Results are accumulated and returned in FP32 (IEEE 754 single precision).

Forward packet (master to worker, for one output row): one routing flit, one tag flit, 128 Q8_0 blocks $\times$ (1 scale flit + 8 weight flits), and 2048 BF16 activation flits, totaling $P_{\text{fwd}} = 3201$ payload flits.

Return packet: one routing flit, one tag flit, and one FP32 result flit — $P_{\text{ret}} = 2$ payload flits, identical to the latency experiment.

Computation model. Each of the 15 worker nodes is assigned $14336/16 = 896$ output rows; the master at (0, 0) also computes 896 rows locally. Workers decode Q8_0 weights to FP32 and accumulate the dot product in FP32. Local computation is modeled as instantaneous: injecting one forward packet takes 3202 clock cycles (one per flit), which already exceeds the dot-product budget of a real VLIW unit over 4096 elements, so arithmetic is not the bottleneck. All 14 336 results are verified for exact FP32 equality against a software reference.

Throughput. The simulation completes in **43 102 246 cycles** (dual-clock FIFO, SINGLE_CLOCK_ROUTER=0) with the master’s single egress port sustaining near-100% injection throughout. Link utilization (Table VIII) shows the consequence of this sustained injection: the first East-bound segment of row 0 reaches 79.9%, carrying packets bound for all 12 workers that require at least one East hop. Utilization drops in proportion to the number of workers remaining on each subsequent segment. South-bound links absorb $\frac{1}{4}$ of the total traffic each. Return-path links (West and North) are negligible at $<0.1\%$. The overall average of **6.7%** reflects the single-port injection bottleneck: 47 of 48 directed links are underutilised while one port saturates. Single-clock mode (SINGLE_CLOCK_ROUTER=1) would reduce the cycle count by factor $9/12 = 0.75$ to approximately 32M cycles, as predicted by the validated latency model (8); the $5\times$ speedup of the four-master configuration is mode-independent since both configurations use the same clock setting.

TABLE VIII
LINK UTILIZATION (%) BY DIRECTION, LLaMA 3 8B SINGLE-MASTER RUN (43 102 246 CYCLES). SEG. k : k -TH SEGMENT FROM MASTER (0, 0).
RETURN-PATH LINKS ($<0.1\%$) OMITTED.

Direction	Seg. 1	Seg. 2	Seg. 3
→East, row 0	79.9	53.2	26.6
→South, col 0	20.0	13.3	6.7
→South, col 1	20.0	13.3	6.7
→South, col 2	20.0	13.3	6.7
→South, col 3	20.0	13.3	6.7
Overall average: 6.7%			

The load on each segment is strictly proportional to the number of destinations reachable through it (East: 12, 8, 4 workers; South per column: 3, 2, 1 workers), with no hot-spots beyond the first East segment. Nonetheless, the bottleneck remains the single injection port, motivating the multi-master extension.

G. Four-Master Quadrant Configuration

The single master at (0, 0) creates an injection bottleneck: all 15 forward packets per round enter through one port, and all result packets return to that same port. We now distribute injection across the four corner nodes by assigning each a 2×2 quadrant.

Table IX lists the assignment. Master m handles $Q = D_{\text{out}}/4 = 3584$ output rows. With $N_W = 3$ workers per

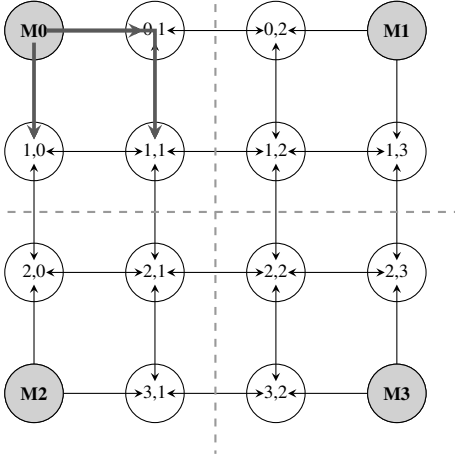


Fig. 16. Four-master 4×4 mesh. Corner nodes M_0 – M_3 (shaded) each coordinate a 2×2 quadrant; dashed lines mark the mesh midpoints. Arrows show M_0 ’s three routes to workers (0, 1), (1, 0), and (1, 1). By symmetry, routes from all four masters remain within their respective quadrant.

quadrant and one row computed locally, each master executes $\text{ROUNDS} = Q/(N_W + 1) = 896$ rounds—where one round consists of sending one forward packet to each of the N_W workers and computing one row locally, then collecting N_W result packets—the same count as the single-master case, but all four masters proceed simultaneously.

TABLE IX
QUADRANT ASSIGNMENT: MASTERS, WORKERS, AND ROW RANGES.

Master	Position	Workers	Rows
M_0	(0, 0)	(0, 1), (1, 0), (1, 1)	0–3583
M_1	(0, 3)	(0, 2), (1, 3), (1, 2)	3584–7167
M_2	(3, 0)	(3, 1), (2, 0), (2, 1)	7168–10751
M_3	(3, 3)	(3, 2), (2, 3), (2, 2)	10752–14335

The structural property exploited here is *link disjointness*: every XY route from a corner master to its three workers stays within the 2×2 quadrant. No packet crosses the horizontal midline (between rows 1 and 2) or the vertical midline (between columns 1 and 2), so the four quadrants are traffic-isolated by construction, without any runtime coordination.

Figure 16 illustrates the topology and the three routes from M_0 : one East hop to (0, 1), one South hop to (1, 0), and East+South to (1, 1).

Computation model. Each master computes one row locally per round (row $mQ + t(N_W + 1)$ in round t , for $t = 0, \dots, 895$), for a total of 896 local rows per master and 3584 across all four. Local dot products are modeled as instantaneous: a VLIW unit over $D_{\text{in}} = 4096$ elements completes well within the 3×3202 clock-cycle injection window per round, so arithmetic is not on the critical path. All 14 336 results are verified for exact FP32 equality against a software reference.

H. Throughput Scaling Results

The four-master simulation completes in **8 620 484 cycles**, a **5.0 $\times$** speedup over the 43 102 246 cycles of the single-master

run (Table X).

TABLE X
SINGLE- VS. FOUR-MASTER THROUGHPUT ($D_{\text{OUT}} = 14336$, $P_{\text{FWD}} = 3201$ PAYLOAD FLITS).

Configuration	Masters	Cycles	Speedup
Single master	1	43 102 246	1 $\times$
Four masters	4	8 620 484	5 $\times$

The 5 $\times$ gain exceeds the 4 $\times$ expected from simple injection parallelism. The additional margin comes from backpressure relief: each corner master serves only three workers, whereas the single master drives fifteen. With fewer concurrent result packets converging on one egress port, injection stalls are shorter.

Table XI reports the utilization pattern. The quadrant isolation property is confirmed quantitatively: every link crossing either mesh midpoint registers exactly 0%. Within each quadrant, the first East or West hop from the corner master reaches 66.6% because two of the three forward routes share it (the direct worker and the diagonal worker); each South or North second-hop segment carries one route at 33.3%. By symmetry all four quadrants show identical figures. The overall average of 11.1% contrasts with the 3.3% of the single-master run, reflecting simultaneous injection from four ports and correspondingly more uniform mesh loading.

TABLE XI
REPRESENTATIVE LINK UTILIZATION (%), FOUR-MASTER RUN (8 620 484 CYCLES). BY SYMMETRY ALL FOUR QUADRANTS ARE IDENTICAL; CROSS-BOUNDARY LINKS ARE EXACTLY 0%.

Direction	Segment	Role	Util (%)
→East, row 0	(0, 0) → (0, 1)	2/3 fwd. routes from M_0	66.6
→West, row 0	(0, 3) → (0, 2)	2/3 fwd. routes from M_1	66.6
→East, row 3	(3, 0) → (3, 1)	2/3 fwd. routes from M_2	66.6
→West, row 3	(3, 3) → (3, 2)	2/3 fwd. routes from M_3	66.6
→South, col 0	(0, 0) → (1, 0)	direct South worker of M_0	33.3
→South, col 1	(0, 1) → (1, 1)	diagonal worker of M_0	33.3
→North, col 0	(3, 0) → (2, 0)	direct North worker of M_2	33.3
→North, col 1	(3, 1) → (2, 1)	diagonal worker of M_2	33.3
Cross-boundary	(any)	—	0.0
Overall average			11.1

All results in this section are obtained from Verilator co-simulation; on-device measurement on physical FPGA hardware remains open and is identified as future work in Section VIII.

VIII. CONCLUSION

HyNoC is a hybrid circuit-switch/wormhole Network-on-Chip designed for distributed VLIW computing on FPGA. By combining source routing—which establishes a dedicated path through a distributed request/grant handshake—with wormhole flit streaming, HyNoC achieves bounded, deterministic transfer latency without the area overhead of virtual channels.

Congestion is managed statically by the compiler or system software, which selects routes across the rich set of shortest

paths available in mesh and higher-dimensional topologies. The combinatorial analysis presented in Section IV shows that enriching the physical topology is more area-efficient than adding virtual channels for the targeted workload class, provided that a compile-time scheduler distributes traffic across available paths.

The parallel round-robin arbiter provides starvation-free, fixed-latency arbitration at each egress port, and per-port independent clock domains allow the NoC to interface cleanly with heterogeneous processing elements running at different frequencies. The multiple local interface mechanism offers a practical alternative to virtual channels when a node requires simultaneous non-blocking communication with multiple peers.

Verilator co-simulation on a 4×4 mesh validates the analytical latency model: measured return-path latency satisfies $L = 12H + P - 1$ (dual-clock) and $L = 9H + P - 1$ (single-clock), with the $3H$ gap between modes explained exactly by the CDC pipeline overhead eliminated in single-clock operation. A LLaMA 3 8B FFN up-projection benchmark (Q8_0 weights, BF16 activations, 14336×4096) with a single master completes in 43 M cycles; a four-master quadrant configuration, where each corner master owns a 2×2 traffic-isolated quadrant, reduces this to 8.6 M cycles—a $5 \times$ speedup with zero cross-quadrant link utilization, confirming the deterministic throughput scaling predicted by the link-disjoint topology design.

HyNoC is released as open-source hardware under the CERN-OHL-P v2 license, providing a complete RTL implementation with simulation testbenches and FPGA board support.

Future work includes: formal deadlock-freedom analysis for arbitrary source route sets; synthesis results and latency characterization on representative FPGA devices (Section VII provides Verilator-based characterization; on-device measurement remains open); integration with a VLIW compiler backend for automatic route assignment and injection scheduling; and extension of the XY routing protocol placeholder to a full hardware implementation.

ACKNOWLEDGMENT

The author used Claude Sonnet 4.6 (Anthropic) as an AI writing assistant during the preparation of this manuscript, including for drafting and revising text, literature analysis, and L^AT_EX editing. All technical content, claims, and results have been verified by the author.

REFERENCES

- [1] W. J. Dally and B. Towles, “Route packets, not wires: On-chip interconnection networks,” in *Proc. Design Automation Conference (DAC)*, 2001, pp. 684–689.
- [2] W. J. Dally, “Virtual-channel flow control,” *IEEE Transactions on Parallel and Distributed Systems*, vol. 3, no. 2, pp. 194–205, 1992.
- [3] A. Mello, L. Tedesco, N. Calazans, and F. Moraes, “Virtual channels in networks on chip: Implementation and evaluation on Hermes NoC,” in *Proc. 18th Annual Symposium on Integrated Circuits and System Design (SBCCI)*, 2005, pp. 178–183.
- [4] F. Moraes, N. Calazans, A. Mello, L. Möller, and L. Ost, “HERMES: an infrastructure for low area overhead packet-switching networks on chip,” *Integration, the VLSI Journal*, vol. 38, no. 1, pp. 69–93, 2004.
- [5] C. Clienti, “Veriparse: A source-to-source transformation toolkit for verilog/systemverilog,” <https://github.com/cclienti/veriparse>, 2024, IGPLv3 license.
- [6] W. J. Dally and B. Towles, *Principles and Practices of Interconnection Networks*. Morgan Kaufmann, 2004.
- [7] W. J. Dally and C. L. Seitz, “Deadlock-free message routing in multiprocessor interconnection networks,” vol. 36, no. 5, 1987, pp. 547–553.
- [8] S. Kumar *et al.*, “A network on chip architecture and design methodology,” in *Proc. IEEE Computer Society Annual Symposium on VLSI (ISVLSI)*. IEEE, 2002, pp. 105–112.
- [9] K. Goossens, J. Dielissen, and A. Radulescu, “Æthereal network on chip: Concepts, architectures, and implementations,” *IEEE Design & Test of Computers*, vol. 22, no. 5, pp. 414–421, 2005.
- [10] J. Duato, “A new theory of deadlock-free adaptive routing in wormhole networks,” *IEEE Transactions on Parallel and Distributed Systems*, vol. 4, no. 12, pp. 1320–1331, 1993.
- [11] M. Liwei and S. Yihe, “On-chip network design automation with source routing switches,” vol. 12, no. 1, 2007, pp. 77–85.
- [12] S. Mubeen and S. Kumar, “Designing efficient source routing for mesh topology network on chip platforms,” in *Proc. 13th Euromicro Conf. on Digital System Design (DSD)*, 2010, pp. 181–188.
- [13] B. D. de Dinechin and A. Graillat, “Feed-forward routing for the wormhole switching Network-on-Chip of the Kalray MPPA2-256 processor,” in *Proc. 10th International Workshop on Network on Chip Architectures (NoCArc)*, 2017.
- [14] C. J. Glass and L. M. Ni, “The turn model for adaptive routing,” in *Proc. International Symposium on Computer Architecture (ISCA)*, 1992, pp. 278–287.
- [15] M. K. Papamichael and J. C. Hoe, “CONNECT: Re-examining conventional wisdom for designing NoCs in the context of FPGAs,” in *Proc. ACM/SIGDA International Symposium on FPGAs (FPGA)*, 2012, pp. 37–46.
- [16] N. Kapre and J. Gray, “Hoplite: Building austere overlay NoCs for FPGAs,” in *Proc. 25th International Conference on Field Programmable Logic and Applications (FPL)*, 2015, pp. 1–8.
- [17] S. M. Nair *et al.*, “HopliteRT: A real-time aware NoC for FPGAs,” in *Proc. International Conference on Field Programmable Logic and Applications (FPL)*, 2019.
- [18] I. Swarbrick, D. Gaitonde, S. Ahmad, B. Gaide, and Y. Arbel, “Network-on-chip programmable platform in Versal ACAP architecture,” in *Proc. ACM/SIGDA International Symposium on Field-Programmable Gate Arrays (FPGA)*, 2019, pp. 212–221.
- [19] J. Pontes, M. Moreira, F. Moraes, and N. Calazans, “Hermes-A: An asynchronous NoC router with distributed routing,” in *Proc. PATMOS*, 2011, pp. 150–159.
- [20] J. R. Ellis, “Bulldog: A compiler for VLIW architectures,” Ph.D. dissertation, Yale University, 1986.
- [21] B. de Dinechin *et al.*, “A clustered manycore processor architecture for embedded and accelerated applications,” in *Proc. IEEE High Performance Extreme Computing (HPEC)*, 2013, pp. 1–6.
- [22] ARM Limited, *AMBA AXI and ACE Protocol Specification*, ARM Limited, 2011, iHI0022D, version D.
- [23] D. E. Knuth, *The Art of Computer Programming, Volume 1: Fundamental Algorithms*, 3rd ed. Addison-Wesley, 1997, section 1.2.5: Permutations and Factorials.
- [24] C. Clienti, “Wavedisp: A python-based waveform display configuration tool,” <https://github.com/cclienti/wavedisp>, 2019, MIT license.
- [25] “CERN open hardware licence version 2 – permissive,” https://ohwr.org/cern_ohl_p_v2.txt, 2020.
- [26] W. Snyder *et al.*, “Verilator: The fastest Verilog/SystemVerilog simulator,” <https://www.veripool.org/verilator/>, 2024, open-source under LGPLv3.
- [27] A. Dubey *et al.*, “The Llama 3 herd of models,” *arXiv preprint arXiv:2407.21783*, 2024. [Online]. Available: <https://arxiv.org/abs/2407.21783>
- [28] G. Gerganov *et al.*, “llama.cpp: LLM inference in plain C/C++,” <https://github.com/ggerganov/llama.cpp>, 2023, open-source under MIT license.